

HW4-2 文档：9×9 乘法表纠错

题目

- 给定一个包含错误的 9×9 乘法表（数据段 81 字节），校验每个位置是否正确，将错误的位置打印到屏幕。
- 要求用 `call/ret` 实现过程调用，注意寄存器在过程调用前后的保存与恢复；使用双重循环；注意数据段寻址方式。
- C 语言实现后反汇编并注释，观察全局/局部数据差别。

实现文件

- `check_table.asm`：16 位 DOS 汇编版本，使用过程 `PRINT_ERROR` 与 `PRINT_BYTE_DEC`。
- `check_table.c`：等价的 C 版本。

数据表（含错误）

按行存放 9×9=81 字节，例如第一行 7,2,3,4,5,6,7,8,9，第二行末尾故意放 0，等等。

汇编实现要点

- 外层循环：`CL` 作为行索引 i (1..9)；内层循环：`BL` 作为列 j (1..9，每行固定 9 列)，`CH` 作为列计数。`SI` 指向当前表项。
- 期望值计算：`AL=i`，`MUL BL` 得到 `AX = i*j`，期望值放到 `DH`，实际值放 `DL`，比较后若不等调用 `PRINT_ERROR`。
- `PRINT_ERROR` 输入：`CL=i`，`BL=j`，`DL=实际值`，`DH=期望值`。过程内成对压栈/出栈 `AX`、`BX`、`CX`、`DX`，逐段输出 `r= i c= j v= 实际 exp= 期望`。
- `PRINT_BYTE_DEC` 将 `AL(0..99)` 输出为两位宽度，十位为 0 时输出空格，便于对齐。过程内保存 `AX`、`BX`、`DX`。
- 结束后输出 `accomplish!`。

C 版要点

- 全局数组 `table[9][9]` 存放同样的数据；双重循环比较 `actual` 与 `(i+1)*(j+1)`。
- 输出格式：`r=X c=Y v=实际 exp=期望`，无错误则打印 `accomplish!`。
- 反汇编可观察：全局数据位于数据段，函数栈帧保存局部变量，循环控制在寄存器/栈上，`printf` 参数按调用约定入栈。

使用

- 汇编版：用 MASM/TASM 组译链接 `check_table.asm`，运行后按行列列出错误项；无交互输入。
- C 版：编译 `check_table.c` 后运行，行为相同。